

Bootcamp: Full Stack Junior

Modulo 4: Profesionalización Fronted con TypeScript y Next.js

Semana 13

Contenido

1

Implementación de TanStack Query

2

Estrategias de Fetching: Secuencial vs. Paralelo (Waterfall mitigation).

3

Mecanismos de invalidación y revalidación de caché en Next.js.

Contenido:

1. Implementación de TanStack Query

1.1. El Problema: ¿Por qué useEffect ya no es suficiente?

En React tradicional, el manejo de datos asíncronos se realiza combinando `useEffect` + `useState`. Esto funciona, pero rápidamente escala en complejidad: debemos manejar estados de carga, errores, reintentos, sincronización, caché manual, invalidaciones, refetching y consistencia global.

En aplicaciones reales (e-commerce, dashboards, SaaS), esta estrategia se vuelve frágil y repetitiva. Aquí es donde entra TanStack Query (antes React Query): una librería especializada en Server State Management.

TanStack Query no es solo para hacer `fetch`... es un **gestor inteligente de estado remoto con caché automática, sincronización y control fino del ciclo de vida de datos**.

Manejo Manual (useEffect)	TanStack Query
Manejo manual de loading/error	Estados automáticos
Sin caché por defecto	Caché inteligente
Refetch manual	Refetch automático
Código repetitivo	Declarativo y limpio
Difícil invalidación	Invalidación controlada

Tabla 1 - Comparativa Técnica - Fuente: Autor de Contenidos

Ejemplo Manual (Anti-Patrón Escalable)

```

import { useEffect, useState } from "react";

interface Product {
  id: number;
  title: string;
}

export function ProductList() {
  const [data, setData] = useState<Product[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    setLoading(true);

    fetch("https://api.example.com/products")
      .then(res => res.json())
      .then(setData)
      .catch(() => setError("Error fetching products"))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>{error}</p>;

  return (
    <ul>
      {data.map(product => (
        <li key={product.id}>{product.title}</li>
      ))}
    </ul>
  );
}

```

Ilustración 1 - Fetch tradicional en Reactjs- Fuente: Autor de contenidos

No uses useEffect para manejar estado remoto complejo. useEffect no fue diseñado como gestor de caché ni sincronización distribuida.

1.2. Instalación y Configuración del QueryClient

TanStack Query funciona mediante un **QueryClient**, que es el cerebro del sistema. Este cliente:

- Administra el caché global
- Controla reintentos
- Define tiempos de stale
- Orquesta invalidaciones

El QueryClientProvider expone ese cliente a toda la aplicación.

Instalación

```
pnpm add @tanstack/react-query
```

Configuración en Next.js 16+ (App Router)

providers.tsx

```
"use client";

import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { ReactNode, useState } from "react";

interface ProvidersProps {
  children: ReactNode;
}

export function Providers({ children }: ProvidersProps) {
  // useState evita recrear el cliente en cada render
  const [queryClient] = useState(() =>
    new QueryClient({
      defaultOptions: {
        queries: {
          staleTime: 1000 * 60, // 1 minuto
          retry: 1,
        },
      },
    })
  );

  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  );
}
```

Ilustración 3 – provider TanStack Query- Fuente: Autor de contenidos

Layout.tsx

```
import { Providers } from "../providers";

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

Ilustración 4 – layout principal - Fuente: Autor de contenidos

Nunca crees el QueryClient fuera de un useState o useMemo en entornos client-side de

Next.js. Podrías romper el caché en cada render.

1.3. Fetching de Datos con useQuery

Los 3 Pilares

1. **queryKey** → Identificador único
2. **queryFn** → Función async
3. **Return State** → data, isLoading, error, isFetching, etc.

Beneficios de queryKey

- Identificación única
- Soporte para parámetros dinámicos
- Invalidación selectiva
- Caché automática

```
"use client";

import { useQuery } from "@tanstack/react-query";

interface Product {
  id: number;
  title: string;
  price: number;
}

async function fetchProducts(): Promise<Product[]> {
  const res = await fetch("https://api.example.com/products");

  if (!res.ok) {
    throw new Error("Error fetching products");
  }

  return res.json();
}

export function ProductList() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["products"],
    queryFn: fetchProducts,
  });

  if (isLoading) return <p>Cargando...</p>;
  if (error instanceof Error) return <p>{error.message}</p>;

  return (
    <ul>
      {data?.map(product => (
        <li key={product.id}>
          {product.title} - ${product.price}
        </li>
      ))}
    </ul>
  );
}
```

Ilustración 5 – Ejemplo profesional useQuery - Fuente: Autor de contenidos

1.4. Mutaciones con useMutation

¿Cuándo usar mutaciones?

Para operaciones de escritura:

- POST (crear)
- PUT/PATCH (actualizar)
- DELETE (eliminar)

A diferencia de `useQuery`, las mutaciones no se ejecutan automáticamente.

<code>useQuery</code>	<code>useMutation</code>
GET (lectura)	POST/PUT/DELETE
Ejecuta automáticamente	Ejecuta manualmente
Cacheable	No cacheable directamente
Basado en <code>queryKey</code>	Basado en acción

Tabla 2 - `useQuery` vs `useMutation` - Fuente: Autor de Contenidos

```
"use client";

import { useMutation, useQueryClient } from "@tanstack/react-query";

interface NewProduct {
  title: string;
  price: number;
}

async function createProduct(data: NewProduct) {
  const res = await fetch("https://api.example.com/products", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });

  if (!res.ok) {
    throw new Error("Error creating product");
  }

  return res.json();
}

export function CreateProductForm() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: createProduct,
    onSuccess: () => {
      // Refresca automáticamente la lista
      queryClient.invalidateQueries({ queryKey: ["products"] });
    },
  });

  return (
    <form
      onSubmit={(e) => {
        e.preventDefault();
        mutation.mutate({ title: "Nuevo Producto", price: 99 });
      }}
    >
      <button disabled={mutation.isPending}>
        {mutation.isPending ? "Creando..." : "Crear Producto"}
      </button>
    </form>
  );
}
```

Ilustración 6 – Ejemplo profesional `useMutation` - Fuente: Autor de contenidos

2. Estrategias de Fetching: Secuencial vs. Paralelo

Imagina que estás en un restaurante.

Si el mesero toma tu orden de bebida, va a cocina, espera que la preparen... luego regresa para tomar el plato fuerte... luego espera... estás ante un proceso **secuencial**.

Ahora imagina que el mesero toma TODO tu pedido al mismo tiempo y cocina empieza a preparar todo en paralelo. Mucho más eficiente.

En aplicaciones web ocurre lo mismo.

La forma en que pedimos los datos impacta directamente la **velocidad percibida**.

Muchas veces una app se siente lenta no por el internet... sino por cómo estructuramos nuestras peticiones.

Criterio	Fetching Secuencial	Fetching Paralelo
Velocidad	Más lento	Más rápido
Complejidad	Simple	Moderada
Casos de uso	Dependencias entre datos	Datos independientes
Riesgo	Waterfall	Manejo de errores combinados

Tabla 4 - Tipos de fetching - Fuente: Autor de Contenidos

2.1. Fetching Secuencial y el Problema del “Waterfall”

¿Qué es un Waterfall?

Un **Waterfall de red** ocurre cuando:

- Una petición espera que termine la anterior
- El navegador ejecuta tareas una tras otra
- Se desperdicia tiempo mientras podría trabajar en paralelo

Es como una fila única en el banco cuando hay 5 cajas disponibles.

- El problema no es la red.
- El problema es nuestra lógica.

```
// Simulación en Next.js Server Component o API Route

async function getUser() {
  const res = await fetch("https://api.example.com/user");
  return res.json();
}

async function getOrders() {
  const res = await fetch("https://api.example.com/orders");
  return res.json();
}

export default async function Page() {
  const user = await getUser(); // Espera 1
  const orders = await getOrders(); // Espera 2

  return (
    <div>
      <h1>{user.name}</h1>
      <p>Órdenes: {orders.length}</p>
    </div>
  );
}
```

Ilustración 7 – Ejemplo que genera Waterfall - Fuente: Autor de contenidos

Si cada petición tarda 1 segundo, total = 2 segundos.

2.2. Fetching Paralelo (Optimización)

Si los datos son independientes, podemos pedirlos al mismo tiempo.

Promise.all

Promise.all permite ejecutar múltiples promesas en paralelo y esperar a que todas terminen.

```
async function getUser() {
  const res = await fetch("https://api.example.com/user");
  return res.json();
}

async function getOrders() {
  const res = await fetch("https://api.example.com/orders");
  return res.json();
}

export default async function Page() {
  const [user, orders] = await Promise.all([
    getUser(),
    getOrders(),
  ]);

  return (
    <div>
      <h1>{user.name}</h1>
      <p>Órdenes: {orders.length}</p>
    </div>
  );
}
```

Si cada petición tarda 1 segundo, total = **1 segundo**.

useQueries con TanStack Query

Cuando trabajamos en cliente y tenemos múltiples queries dinámicas, podemos usar TanStack Query.

```
"use client";

import { useQueries } from "@tanstack/react-query";

async function fetchUser() {
  const res = await fetch("/api/user");
  return res.json();
}

async function fetchOrders() {
  const res = await fetch("/api/orders");
  return res.json();
}

export function Dashboard() {
  const results = useQueries({
    queries: [
      { queryKey: ["user"], queryFn: fetchUser },
      { queryKey: ["orders"], queryFn: fetchOrders },
    ],
  });

  const [userQuery, ordersQuery] = results;

  if (userQuery.isLoading || ordersQuery.isLoading) {
    return <p>Cargando...</p>;
  }

  return (
    <div>
      <h1>{userQuery.data.name}</h1>
      <p>Órdenes: {ordersQuery.data.length}</p>
    </div>
  );
}
```

No mezcles múltiples useQuery dependientes sin control.

Si una depende de otra, usa enabled.

3. Mecanismos de invalidación y revalidación de caché

Piensa en la **caché como una despensa de comida en tu casa** .

Si acabas de comprar los ingredientes, están frescos (**Fresh**).

Pero si pasan días, aunque todavía estén ahí, podrían estar vencidos (**Stale**).

En frontend moderno, los datos no son eternos.

Tienen un **ciclo de vida**.

En **TanStack Query**, los datos pasan por diferentes estados hasta ser eliminados de memoria.

Estado	Qué significa	¿Visible al usuario?
Fresh	Datos recientes, no necesitan refetch	Sí
Fetching	Se están actualizando	Sí
Stale	Pueden estar desactualizados	Sí
Inactive	Nadie los está usando	No

Tabla 5 - Estados de la Caché - Fuente: Autor de Contenidos

3.1. TanStack Query – Revalidación Automática

staleTime y gcTime

En TanStack Query existen dos conceptos clave:

- **StaleTime:** Tiempo durante el cual los datos se consideran Fresh.
- **gcTime (antes cacheTime):** Tiempo que los datos permanecen en memoria después de volverse Inactive.

refetchOnWindowFocus

Por defecto, cuando vuelves a la pestaña del navegador, la query puede revalidarse automáticamente.

Esto es útil para dashboards, pero innecesario para contenido estático.

Cuando desactivar revalidación automática

- Formularios largos
- Contenido estático
- APIs costosas
- Sistemas offline-first

Configuración Global del QueryClient

```

"use client";

import {
  QueryClient,
  QueryClientProvider,
} from "@tanstack/react-query";
import { ReactNode, useState } from "react";

interface ProvidersProps {
  children: ReactNode;
}

export function Providers({ children }: ProvidersProps) {
  const [queryClient] = useState(() =>
    new QueryClient({
      defaultOptions: {
        queries: {
          staleTime: 1000 * 60 * 5, // 5 minutos
          gcTime: 1000 * 60 * 10, // 10 minutos
          refetchOnWindowFocus: false,
        },
      },
    })
  );

  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  );
}

```

Ilustración 10 – Configuración de cache QueryClient - Fuente: Autor de contenidos

3.2. Next.js 16+ – Caché en el Servidor

En **Next.js 16+**, el sistema de caché evolucionó.

Existen dos niveles principales:

- **Data Cache:** Cachea el resultado de `fetch()` en Server Components.
- **Full Route Cache:** Cachea el HTML completo de una ruta.

Revalidación basada en tiempo

```

export const revalidate = 60; // 60 segundos

```

Ilustración 11 – Revalidación de cache - Fuente: Autor de contenidos

Esto significa: Después de 60 segundos, la próxima petición generará nueva versión.

Revalidación bajo demanda (Cache Tags)

Permite agrupar fetches bajo una etiqueta.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a single line of JavaScript code:

```
await fetch("https://api.example.com/products", {  
  next: { tags: ["products"] },  
});
```

```
await fetch("https://api.example.com/products", {  
  next: { tags: ["products"] },  
});
```

Ilustración 12 – Cache bajo demanda - Fuente: Autor de contenidos

Recursos de Apoyo



Guía

[Guía de server side rendering avanzado.](#)



Guía

[Guía de como manipular el cache](#)



Video

[Tutorial de ejemplos prácticos con Tanstack query](#)